

Programación Orientada a Objetos en Python

Clases, Herencia y Polimorfismo

Diego Gonzales Edda Martínez

Programación II

24 de noviembre de 2025

¿Qué es la Programación Orientada a Objetos?

Definición

Paradigma que organiza el código en **objetos** que contienen datos (atributos) y comportamientos (métodos).

Ventajas:

- Reutilización de código
- Modularidad
- Mantenibilidad
- Abstracción

Conceptos Clave:

- Clases
- Objetos
- Atributos
- Métodos
- Encapsulamiento

¿Qué es una Clase?

Definición

Una **clase** es un molde o plantilla para crear objetos. Define la estructura (atributos) y el comportamiento (métodos).

Sintaxis básica en Python:

```
class NombreClase:
    def __init__(self):
        # Atributos
        pass

    def metodo(self):
        # Comportamiento
        pass
```

Constructor (`__init__`)

Método especial que se ejecuta automáticamente al crear un objeto. Inicializa los atributos.

Ejemplo:

```
class Estudiante:
    def __init__(self, nombre, edad, carrera):
        self.nombre = nombre # Atributo
        self.edad = edad
        self.carrera = carrera
```

- **self**: Referencia al objeto actual
- **Atributos de instancia**: Únicos para cada objeto

Métodos de Instancia

Definición

Los **métodos** son funciones dentro de una clase que describen el comportamiento de los objetos.

Ejemplo:

```
class Estudiante:
    def __init__(self, nombre, edad, carrera):
        self.nombre = nombre
        self.edad = edad
        self.carrera = carrera

    def presentarse(self):
        return f"Hola, soy {self.nombre}"

    def cumplir_anios(self):
        self.edad += 1
```

Creación de Objetos (Instanciación)

Crear objetos a partir de una clase:

```
# Crear objetos (instancias)
estudiante1 = Estudiante("Diego", 20, "Ingenieria")
estudiante2 = Estudiante("Edda", 21, "Computacion")

# Acceder a atributos
print(estudiante1.nombre)      # Diego
print(estudiante2.carrera)     # Computacion

# Llamar métodos
print(estudiante1.presentarse())

# Output: Hola, soy Diego

estudiante1.cumplir_anios()
print(estudiante1.edad)        # 21
```

Definición

Ocultar detalles internos y exponer solo lo necesario mediante atributos públicos, protegidos o privados.

Ejemplo:

```
class CuentaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular          # Público
        self._numero = "12345"         # Protegido
        self.__saldo = saldo_inicial   # Privado

    def depositar(self, monto):
        if monto > 0:
            self.__saldo += monto
```

Conceptos aprendidos:

- **Clases:** Plantillas para crear objetos
- **Constructor:** Método `__init__` para inicializar
- **Atributos:** Datos que caracterizan al objeto
- **Métodos:** Comportamientos del objeto
- **Encapsulamiento:** Control de acceso a datos

¿Qué es la Herencia?

Definición

La **herencia** permite crear nuevas clases basadas en clases existentes, heredando sus atributos y métodos.

Ventajas:

- Reutilización de código
- Extensibilidad
- Jerarquías lógicas
- Mantenimiento simplificado

Terminología:

- Clase base/padre: Original
- Clase derivada/hija: Hereda

Jerarquía de Vehículos

Vehículo (Clase Base)



Auto Motocicleta Camión

Ejemplo: Auto, Motocicleta y Camión heredan de Vehículo y comparten sus características comunes.

Concepto de Herencia Simple

Estructura Básica

Una clase hija hereda todos los atributos y métodos de la clase padre, y puede agregar los suyos propios.

Elementos clave:

- **Clase Padre:** Define atributos y métodos comunes
- **Clase Hija:** Hereda de la clase padre usando paréntesis
- **super():** Función para acceder a métodos del padre
- **Constructor:** La clase hija puede llamar al constructor del padre

Importante

La clase hija tiene acceso a todo lo público y protegido del padre.

Ejemplo: Persona y Estudiante

Escenario

Un estudiante **ES UNA** persona con atributos adicionales.

Clase Persona (Padre):

- Atributos: nombre, edad
- Método: presentarse()

Clase Estudiante (Hija):

- Hereda: nombre, edad
- Agrega: carrera
- Nuevo método: estudiar()

Beneficio: No repetimos código. Estudiante reutiliza la funcionalidad de Persona.

Sobrescritura de Métodos (Override)

Definición

Una clase hija puede **redefinir** un método heredado para cambiar su comportamiento específico.

Ejemplo: Animales y sus sonidos

- **Animal (Padre):** Método `hacer_sonido()` retorna “Sonido genérico”
- **Perro (Hijo):** Sobrescribe `hacer_sonido()` para retornar “Guau guau”
- **Gato (Hijo):** Sobrescribe `hacer_sonido()` para retornar “Miau miau”

Ventaja

Cada clase hija puede tener su propia implementación del mismo método.

Función `super()` - Extendiendo Funcionalidad

Propósito

`super()` permite llamar métodos de la clase padre y **extender** su funcionalidad en lugar de reemplazarla completamente.

Ejemplo: Empleado y Gerente

- **Empleado:** Calcula pago con solo el salario base
- **Gerente:**
 - Hereda de Empleado
 - Usa `super()` para obtener el pago base
 - Agrega un bono adicional
 - Retorna: salario base + bono

Resultado: Reutilizamos la lógica del padre y agregamos funcionalidad nueva.

Herencia Múltiple

Definición

Una clase puede heredar de **múltiples clases padres** simultáneamente.

Ejemplo: Pato

- **Clase Volador:** Tiene método volar()
- **Clase Nadador:** Tiene método nadar()
- **Clase Pato:** Hereda de Volador y Nadador
- **Resultado:** Pato puede volar() y nadar()

Consideración

Python usa MRO (Method Resolution Order) para resolver conflictos cuando múltiples padres tienen métodos con el mismo nombre.

¿Qué es el Polimorfismo?

Definición

El **polimorfismo** (del griego “muchas formas”) permite que objetos de diferentes clases respondan al mismo método de manera específica para cada clase.

Beneficios:

- Flexibilidad del código
- Interfaces unificadas
- Facilita extensibilidad
- Código más limpio

Idea Central

“Un mismo mensaje, múltiples comportamientos”

Mismo método `area()`, diferentes implementaciones

Círculo

$$\pi \times r^2$$

Rectángulo

$$\text{base} \times \text{altura}$$

Triángulo

$$\frac{\text{base} \times \text{altura}}{2}$$

Mismo método, diferentes implementaciones según el tipo de objeto.

Concepto

Clases hijas heredan métodos de la clase padre y los **sobrescriben** para proporcionar implementaciones específicas.

Ejemplo: Sistema de Animales

- **Clase Animal (Padre):** Método: `hacer_sonido()`
- **Clases Hijas:**
 - **Perro:** `hacer_sonido()` retorna "Guau guau"
 - **Gato:** `hacer_sonido()` retorna "Miau"
 - **Vaca:** `hacer_sonido()` retorna "Muuu"

Ventaja: Podemos tratar todos los animales de manera uniforme, pero cada uno se comporta de forma única.

Duck Typing en Python

Filosofía

“Si camina como un pato y grazna como un pato, entonces es un pato”

Concepto:

- Python no verifica el **tipo** del objeto
- Solo verifica que el objeto tenga el **método** necesario
- No es necesario heredar de una clase común

Ejemplo:

- Clases: Pato, Persona, Robot
- Todas tienen un método: hablar()
- Podemos llamar hablar() en cualquiera sin importar su tipo
- Python ejecuta el método si existe, sin verificar herencia

Sobrecarga de Operadores

Definición

Permitir que operadores matemáticos (+, -, *, /, etc.) funcionen con objetos personalizados.

Métodos especiales (dunder methods):

- `__add__(self, other)`: Para el operador +
- `__sub__(self, other)`: Para el operador -
- `__mul__(self, other)`: Para el operador *
- `__eq__(self, other)`: Para el operador ==
- `__lt__(self, other)`: Para el operador <

Ejemplo: Clase Vector

- Podemos sumar dos vectores: `vector1 + vector2`
- Python llama internamente a `__add__()`

Propósito

Definir una **interfaz común** que las clases hijas deben implementar. No se pueden instanciar directamente.

Características:

- Se usan con el módulo abc (Abstract Base Classes)
- Contienen métodos abstractos que deben ser implementados
- Garantizan que las subclasses tengan ciertos métodos
- Forzan una estructura común

Ejemplo: Clase FiguraGeometrica

- Define método abstracto: area()
- Círculo y Rectángulo deben implementar area()
- No se puede crear un objeto FiguraGeometrica directamente

Conceptos aprendidos:

- **Polimorfismo:** Mismo mensaje, múltiples comportamientos
- **Polimorfismo con herencia:** Sobrescribir métodos
- **Duck typing:** Flexibilidad sin herencia estricta
- **Sobrecarga de operadores:** Personalizar operadores
- **Clases abstractas:** Definir interfaces obligatorias

Encapsulamiento + Herencia +
Polimorfismo

= Código robusto y flexible